

MUSTAFACODE

Lecture 04

Lecture Contents
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 4: Lecture Contents

Lecture Contents

- **Generalization**

jo common antities or method hota hai multiple classes ka unki ek base class banatay hai then all drive classes usay inherit karti hai .

- **Sub typing (extension)**

base class sa inherit karna ka bad drive class ma koch new method or antities add karta hai .

- **Specialization (restriction)**

add a condtion drive class ko base class sa inharit karna ka bad base class ka data pr drive class ma codition laga dana ka aga 18 sa necha hai to error es trha matlab restration laga dana.

- **Overriding** ek aisa same function jo base class ma hai usay drive class ma dobara banay to matlab base class ka fuction ko overrid kar dana overriding khalata hai

- **Abstract classes** ek aisa main parent class jis sa ham or classes bana sakta haii eisa khata hai abstract classes

- **Concrete classes**

drive class ko Concrete class khata hai

Recap – Inheritance

- Derived class, base class ki **saari characteristics inherit karti hai**
 - Inherited characteristics ke ilawa, derived class ki apni **unique characteristics bhi ho sakti hain**
 - Inheritance ka sab se bada faida **reuse (dobara use karna)** hai
-

◆ 04.1 Inheritance se Related Concepts

- Generalization
 - Subtyping (extension)
 - Specialization (restriction)
-

◆ 04.2 Generalization

OO (Object-Oriented) models mein kuch classes ki **common characteristics** hoti hain.

Hum in common features ko ek **nayi class** mein extract karte hain aur phir original classes ko us se **inherit** kar dete hain.

Object model mein bohat se objects hote hain jin mein **common characteristics** hoti hain. In sab ki **attributes aur behavior** ko combine karke ek **single general class** bana di jati hai.

👉 Base class asal mein ek **general class** hoti hai jo derived classes ke common behavior ko represent karti hai.

Is concept ko **Generalization** kehte hain.

◆ Benefits of Generalization

- Yeh **redundancy (repeat hone wali cheezon)** ko kam karta hai
 - **Reusability** provide karta hai
 - System ko **simple (less complex)** banata hai
-

◆ Important Relationship

Generalization mein base aur child classes ke darmiyan:

👉 **"Is a Kind of Relationship" (ya "Is-A relationship")** hona zaroori hota hai

◆ Final Simple Definition

👉 Generalization = Common features ko ek general class mein combine karna aur reuse karna

Example: Line, Circle and Triangle

Line is shape	Circle is shape	Triangle is shape
+-----+	+-----+	+-----+
Line	Circle	Triangle
+-----+	+-----+	+-----+
color	color	color
vertices	vertices	vertices
length	radius	angle
+-----+	+-----+	+-----+
move()	move()	move()
setColor()	setColor()	setColor()
getLength()	computeArea()	computeArea()
+-----+	+-----+	+-----+

```
+-----+
|      Shape      |
+-----+
|  color           |
|  vertices        |
+-----+
|  move()          |
|  setColor()      |
+-----+
```

+-----+	+-----+	+-----+
Circle	Line	Triangle
+-----+	+-----+	+-----+
radius	length	angle
+-----+	+-----+	+-----+
computeArea()	getLength()	computeArea()
+-----+	+-----+	+-----+

◆ Sub-typing & Specialization

Hum kisi existing model mein **nayi class add karna chahte hain**.

Hamare paas pehle se ek **class hierarchy develop ho chuki hoti hai**.

Hum aisi existing class ko dhoondte hain jisme already kuch **required state aur behavior maujood ho**.

Phir hum **nayi class ko us existing class se inherit karte hain** aur us mein apna **unique behavior add kar dete hain**.

◆ 04.3 Sub-typing (Extension)

Sub-typing ka matlab hai ke **derived class, base class ke sath behavior ke hisaab se compatible hoti hai**.

Derived class ke paas base class ki:

- ✓ Saari characteristics hoti hain
 - + Saath hi kuch extra characteristics bhi hoti hain
-

◆ Behavior Compatibility

Behaviorally compatible ka matlab yeh hota hai:

- 👉 Hum base class ki jagah derived class ko use kar sakte hain
 - 👉 Bina kisi problem ke
-

◆ Simple Definition

Sub-typing ka concept yeh hai:

- 👉 "Derived class base class ko safely replace kar sakti hai"

Sub-typing (Extension) - Example



Circle is extending the behavior of Shape by adding:

- radius
- computeCF()
- computeArea()

Student has two extra attributes:

- program
- studyYear

Student also extends behavior by adding:

- study()
- takeExam()

Subtyping aur generalization aapas mein related concepts hain. Subtyping (extension) aur generalization asal mein ek hi cheez ko do different nazariyon se dekhne ka tareeqa hai.

Subtyping mein hum cheezon ko **top se bottom** ki taraf dekhte hain, jab ke generalization mein hum cheezon ko **bottom se top** ki taraf dekhte hain.

◆ 04.4 Specialization (Restriction)

Hum chahte hain ke existing class hierarchy mein ek nayi class add karein jo existing classes se **bohat milti julti ho**, lekin us ka kuch hissa ya behavior **different ya restricted** ho.

Is situation mein hum **Specialization** ka concept use karte hain.

◆ Definition

Specialization ka matlab hai ke:

👉 **Derived class behavior ke hisaab se base class ke sath compatible nahi hoti**

◆ Behavioral Incompatibility

Behavioral incompatibility ka matlab yeh hai:

👉 Hum hamesha **base class ki jagah derived class ko replace nahi kar sakte**

◆ Key Points

- Derived class mein base class ke muqable mein kuch **different characteristics** hoti hain
 - Ya phir kuch **restrictions (limits)** apply hoti hain
 - Yeh sub-typing ke opposite concept hai
-

◆ Example – Specialization (Restriction)

Maan lein humein ek aur class add karni hai **Adult** ki, kisi special requirement ke liye jaise **ID card generation**.

👉 Adult bhi ek **Person** hai

Lekin:

- Adult ki **age 18 se zyada hoti hai**

- Baqi behavior Person class jaisa hi hota hai
-

◆ Wrong Approach ❌

Ek solution yeh ho sakta hai ke:

- 👉 Hum bilkul **nayi class start se banayein**
 - 👉 Aur Person ka saara code dobara likhein
- ❌ Yeh acha tareeqa nahi hai (code duplication)
-

◆ Correct Approach ✅

Behtar solution yeh hai:

- 👉 **Adult class ko Person class se inherit karein**
 - 👉 Aur us mein **age ki restriction laga dein**
-

◆ Final Simple Line

- 👉 **Specialization = Parent class + restrictions / differences**

Specialization Example

```
+-----+
|   Person   |
+-----+
| age : [0..100] |
| ...         |
+-----+
| setAge(a)    |-----> age = a
| ...         |
+-----+
      ▲
      |
      |
+-----+
|   Adult    |
+-----+
| age : [18..100] |
| ...         |
+-----+
| setAge(a)    |-----> if age < 18
| ...         |           error
+-----+                   else age = a
```

Similarly Natural Numbers² are also Integers³
but with restriction:
Natural numbers contain only positive integers.

```
+-----+
| IntegerSet  |
+-----+
| ...         |
+-----+
| add(elem)   |-----> add element to set
| ...         |
+-----+
      ▲
      |
      |
+-----+
| NaturalSet  |
+-----+
| ...         |
+-----+
```

```
| add(elem) |-----> if elem < 1
| ...      |          error
+-----+          else add to set
```

² Natural Numbers:

1, 2, 3, 4, 5 ...

³ Integers:

..., -3, -2, -1, 0, 1, 2, 3 ...

Add method ka behavior base class aur derived class dono mein hota hai, lekin derived class mein us ka behavior **different hota hai**.

Derived class, base class ka original behavior show nahi karti, balki usay **override karke apna naya behavior define karti hai**.

04.5 Overriding

Kabhi kabhi kisi class ko apni base class ka **default behavior change (override)** karne ki zarurat hoti hai.

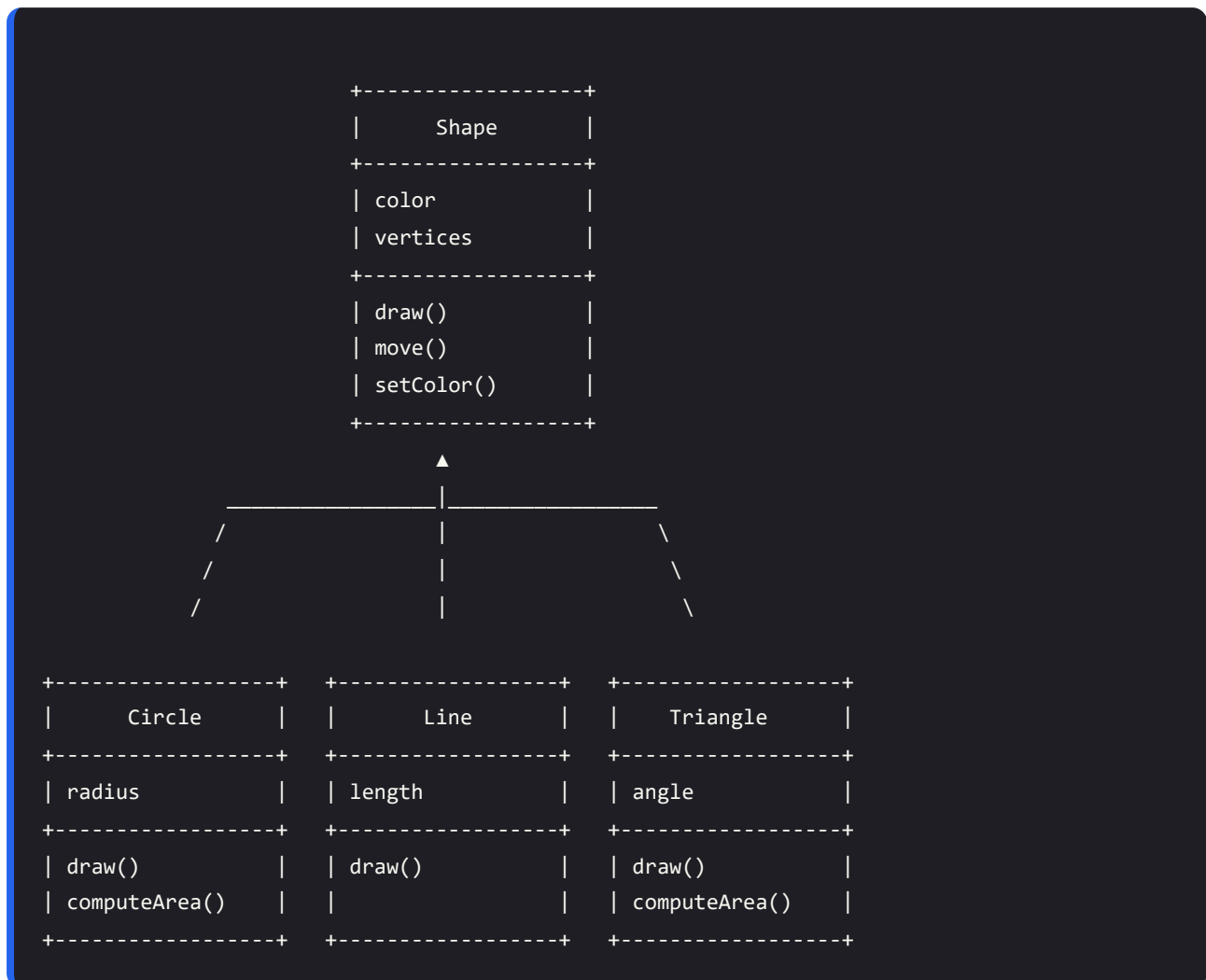
Derived class, base class ke behavior ko **override karke apna naya behavior define karti hai**.

Reasons for Overriding (Overriding kyun use hota hai)

- Derived class ke liye specific behavior dena (specialization)
- Default behavior ko extend karna (extension)
- Default behavior ko restrict karna (restriction)
- Performance improve karna

Overriding ka use **inheritance ko implement karne ke liye** kiya jata hai.

Example – Specific Behaviour (Specialization)



04.6 Abstract Classes

Hamare examples mein hum ne **Shape** aur **Person** ki classes banayi thi. Yeh dono **abstract concepts** hain.

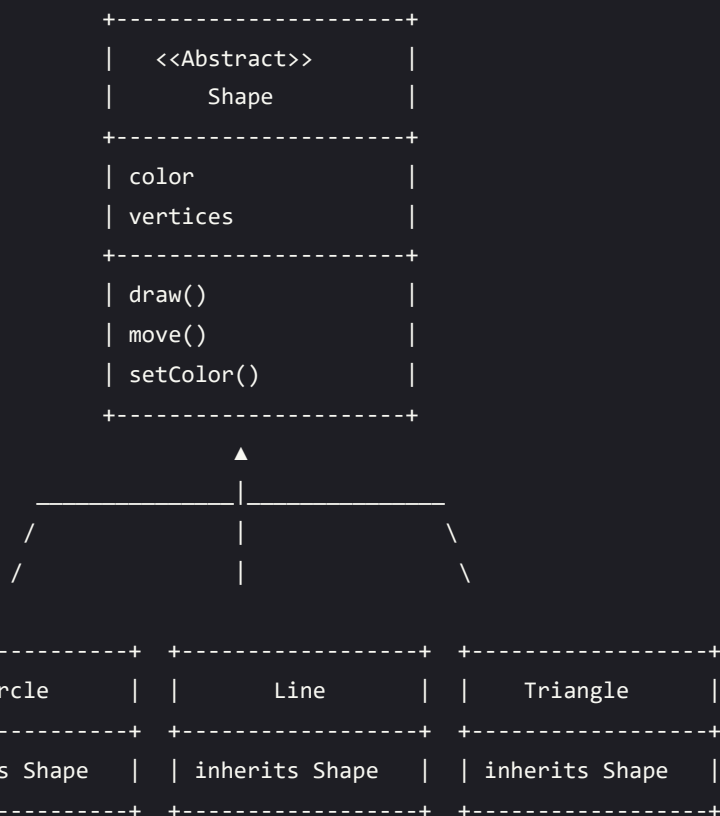
Jo classes hum abstract concepts ke liye banate hain unhein **abstract classes** kehte hain.

Yeh classes aksar **class hierarchy ke top ya us ke qareeb hoti hain** taake sab se **generalized behavior** ko represent kar sakein.

Abstract Class kya karti hai?

- Yeh ek **abstract concept ko implement karti hai** • Is ka main purpose hota hai ke isay **dosri classes inherit karein** • Is ka **object (instance) nahi banaya ja sakta** • Yeh **code reuse (dobara use)** ko promote karti hai

Abstract Classes - Example I



Notes:

- Shape is an abstract class.
- Shape defines common data: color, vertices.
- Shape defines common behavior: move(), setColor().
- draw() is usually implemented by child classes.
- Circle, Line, and Triangle are concrete classes.

Abstract classes **object model** mein akelay exist nahi kar sakti.

Jab hum object model banate hain to sab se pehle hum **objects identify karte hain**, phir un objects mein jo **common attributes hote hain** unhein nikal kar ek **general class** bana dete hain jo class hierarchy ke **top par hoti hai**.

04.7 Concrete Classes

Jo cheezein hum real world mein **actually dekh sakte hain**, unhein **concrete objects** kehte hain. Aur in objects ke liye jo classes banayi jati hain unhein **concrete classes** kehte hain.

Concrete Class kya karti hai?

- Yeh ek **concrete concept ko implement karti hai** • Inka use program mein **objects (instances) banane ke liye hota hai** • Yeh **implementation details provide karti hain** jo specific hoti hain kisi particular domain ke liye

Concrete Classes (Continue)

Ek **concrete class object model mein independently exist kar sakti hai**.

Concrete classes aksar **class hierarchy ke top se neeche (below the top)** hoti hain ek achay object model mein.

Agar koi **abstract class** maujood ho, to object model mein **hierarchy zaroor hoti hai**, kyun ke us abstract class se kuch **concrete classes derived hoti hain**.

Agar concrete classes hi na hon, to abstract class ka koi faida nahi hota.

Glossary

a. Natural Numbers: Numbers jo 1 se start hotay hain aur aagay tak jate hain (1, 2, 3, ...)

b. Integers: Sab positive aur negative numbers aur zero bhi (... -3, -2, -1, 0, 1, 2, 3 ...)

c. Whole Numbers: Numbers jo 0 se start hotay hain (0, 1, 2, 3, ...) Yeh basically natural numbers hotay hain lekin **0 included hota hai**

Kabhi kabhi whole numbers ko **fractional part ke baghair numbers** bhi kaha jata hai.